Name: Rimsha Sohail

Batch: 5

Position: Shine Intern

Domain: Data Science

Table of Contents

Karachi AQI Forecasting System 2

Project Objective 2

Project Overview 3

Project Steps 3
Project Timeline 5
Description of each step 5
1. Data Collection 6
2. Data Preprocessing 6
3. Feature Engineering 6
4. Exploratory Data Analysis (EDA) 6
Time - Series Plot Outputs 7
Correlation Heatmap 7
Average AQI by Day of Week 8
Average AQI by Hours of Day 8
5. Model Training & Evaluation 8
6. MODEL TESTING 17
7. Feature Store & Model Registry 17
8. API Development 21
9. Dashboard 22

..... 6

Karachi AQI Forecasting System

Project Objective

To develop a real-time AQI prediction system that fetches raw pollutant and weather data, computes engineered features, trains multiple ML models, and provides forecasted air quality insights via an interactive dashboard. The system is built with a scalable and automated ML pipeline and provides hourly updates, explainability via SHAP, and hazard alerts.

Project Overview

Air pollution is one of the most pressing environmental challenges in urban areas, significantly affecting public health, climate, and the overall quality of life. In cities like Karachi, rapid industrialization, traffic congestion, and limited environmental regulation have led to deteriorating air quality. The **Air Quality Index (AQI)** serves as a standardized measure to assess and communicate the level of air pollution in a region. Real-time AQI prediction plays a vital role in empowering authorities and citizens to make timely decisions, reduce exposure to harmful pollutants, and plan interventions effectively.

This project aims to develop an end-to-end, scalable AQI forecasting system using real-time data ingestion, machine learning models, and automation tools. The system fetches weather and pollutant data from external APIs such as **AQICN, IQAir** and **OpenWeather**, processes this raw data into engineered features, and uses these features to train and evaluate forecasting models. These models are capable of predicting current and future AQI levels with the objective of issuing alerts when hazardous conditions are detected.

The project pipeline integrates essential components of a modern ML system, including feature engineering, model training and evaluation, feature store, model registry, and CI/CD pipelines using tools like **GitHub Actions** or Apache Airflow. Predictions are served via a **FastAPI** or **Flask** backend and visualized through an interactive **Streamlit** or **Gradio** dashboard, offering users a real-time view of AQI levels and forecasts.

Additionally, the system leverages SHAP for model explainability, ensuring transparency in prediction decisions. An alert mechanism is incorporated to notify users of various AQI levels, enabling them to take proactive health and safety measures.

This comprehensive solution demonstrates how machine learning, automation, and cloud technologies can be combined to address critical urban challenges like air pollution, ultimately contributing to public awareness and environmental monitoring.

Project Steps

The Data Science Steps are as follows:

- Data Collection

- Real-time weather and air pollutant data were collected using external APIs such as **AQICN, IQAir** and **OpenWeather**.

- Data Preprocessing

- The collected data was cleaned by handling missing values, removing unnecessary columns (like forecast fields), and formatting the time-related fields for consistency.

- Feature Engineering

Meaningful features were created to improve model performance, including:

- Time-based features such as hour, day, and month
- Derived features such as AQI change rate, pollutant ratios, rolling averages, and weather features like temperature and humidity**.**

- Exploratory Data Analysis (EDA)

- The data was analyzed to identify trends, seasonal patterns, distributions, and correlations between pollutants and AQI values.

- Modeling

- Several machine learning and deep learning models were trained and evaluated, including:
  - **Scikit-learn models**: XGBoost , Linear Regression, Ridge Regression
  - **Deep learning models**: LSTM using TensorFlow or PyTorch
- The models were validated using **TimeSeriesSplit**, and evaluated using metrics such as **MAE**, **RMSE**, and **R^2 score**.

- Model Deployment

- The best-performing model was saved in a model registry and deployed through an API using **FastAPI**.

- CI/CD Automation

- Continuous integration and deployment pipelines were created using **GitHub Actions**, enabling:
    - Hourly execution of the feature engineering script ○ Daily execution of the model retraining script

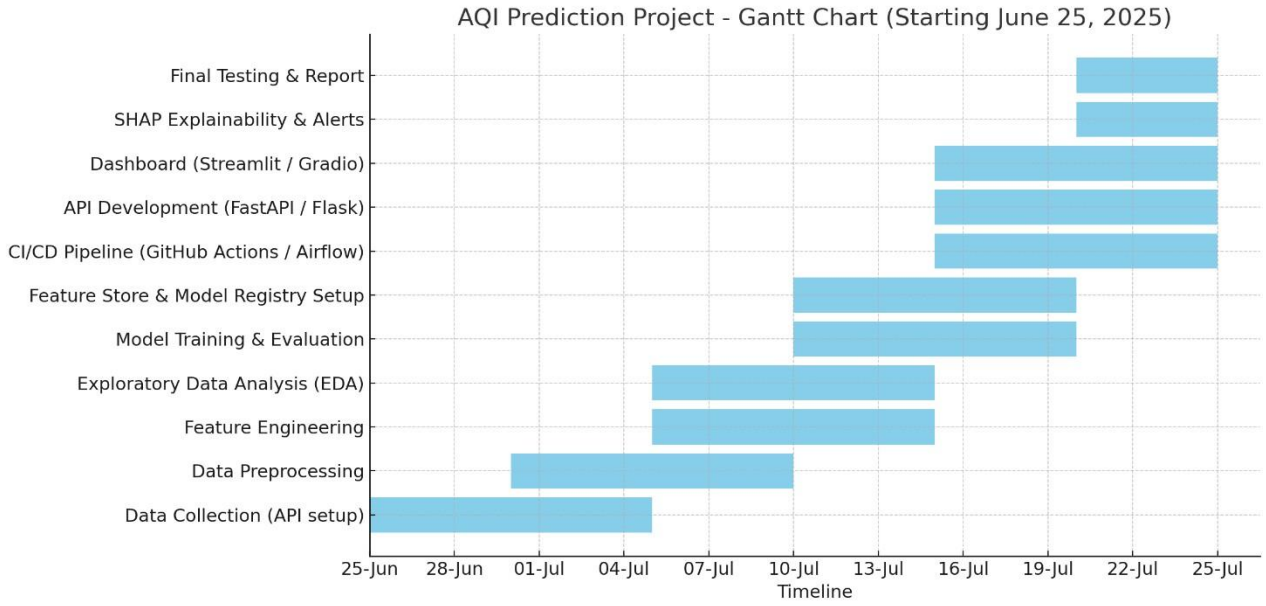
- Model Explainability & Alerts

- **SHAP** was used to explain model predictions and understand feature importance.
- An **alert mechanism** was added to notify users when AQI levels exceed hazardous thresholds.

- Interactive Dashboard

- A real-time dashboard was built using **Gradio** to display:
    - Current AQI
    - 3-day AQI forecast ○ Alert levels and health-related advisories

Project Timeline



Description of each step

1. **Data Collection**

This step involves gathering real-time weather and air quality data of Karachi from external APIs such as AQICN and OpenWeather. The collected data includes pollutants (e.g., PM2.5, PM10, CO, NO_2, etc.) and meteorological parameters (e.g., temperature, humidity, pressure).

2. Data Preprocessing

In this stage, the raw data is cleaned and prepared for modeling. This includes handling missing values, removing irrelevant columns, formatting timestamps, and ensuring data consistency. Additional steps include visualizing null values using heatmaps to identify data gaps, and retrieving all column names to understand the dataset structure before processing begins.

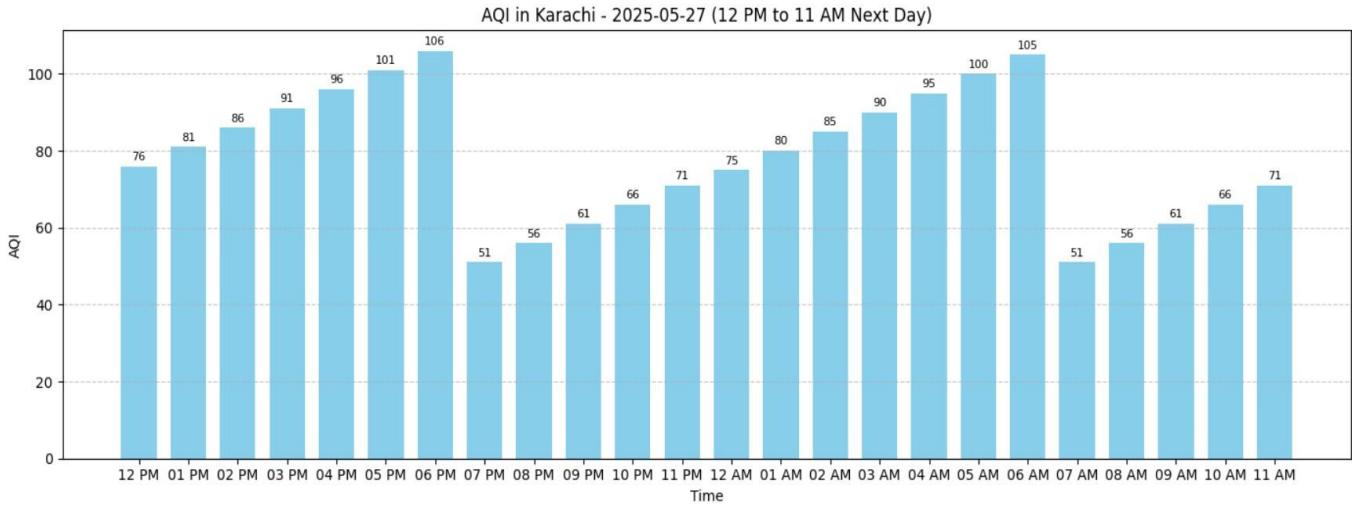
3. Feature Engineering

New features are created to enhance model performance. These include timebased features (hour, day, month), derived features (AQI change rate, pollutant ratios), and weather features (temperature, humidity). These engineered features serve as inputs to the prediction models.

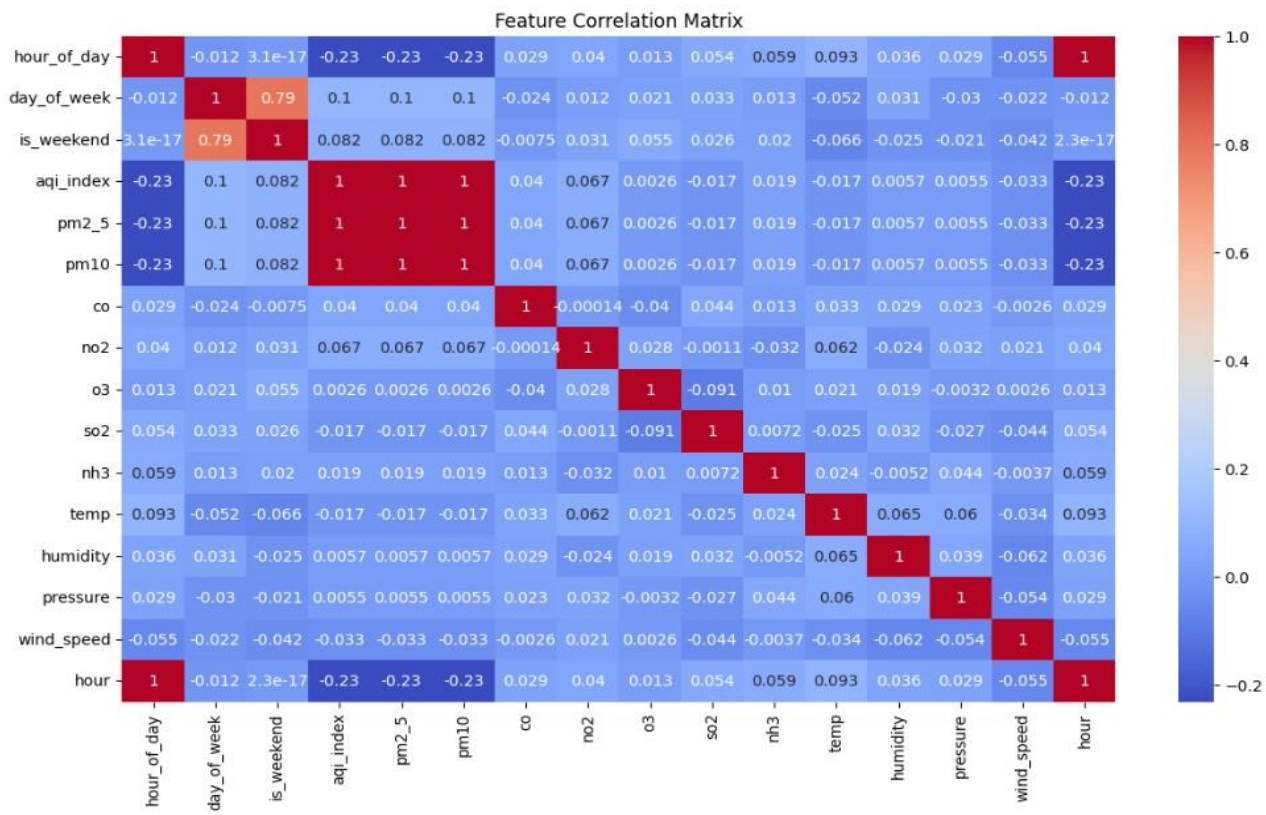
4. Exploratory Data Analysis (EDA)

EDA is used to understand the underlying patterns and relationships in the data. Visualizations such as time-series plots, correlation heatmaps, and distribution graphs are used to gain insights into pollutant behavior and AQI trends.

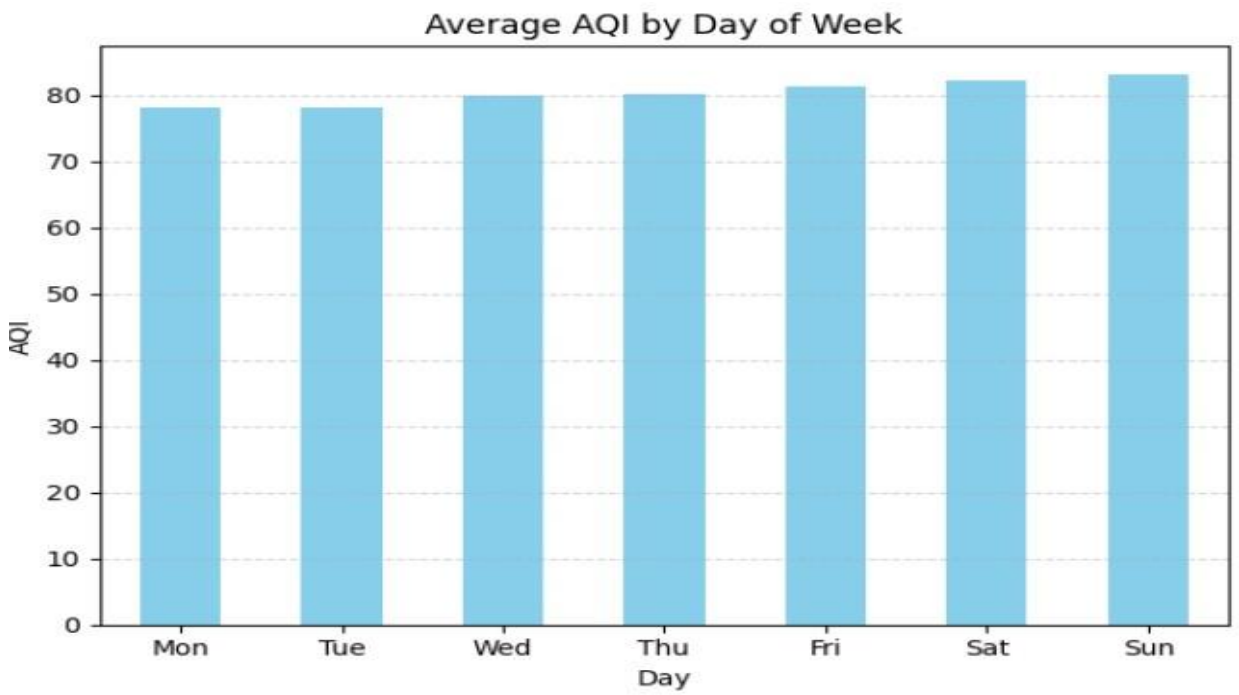
Time - Series Plot Outputs



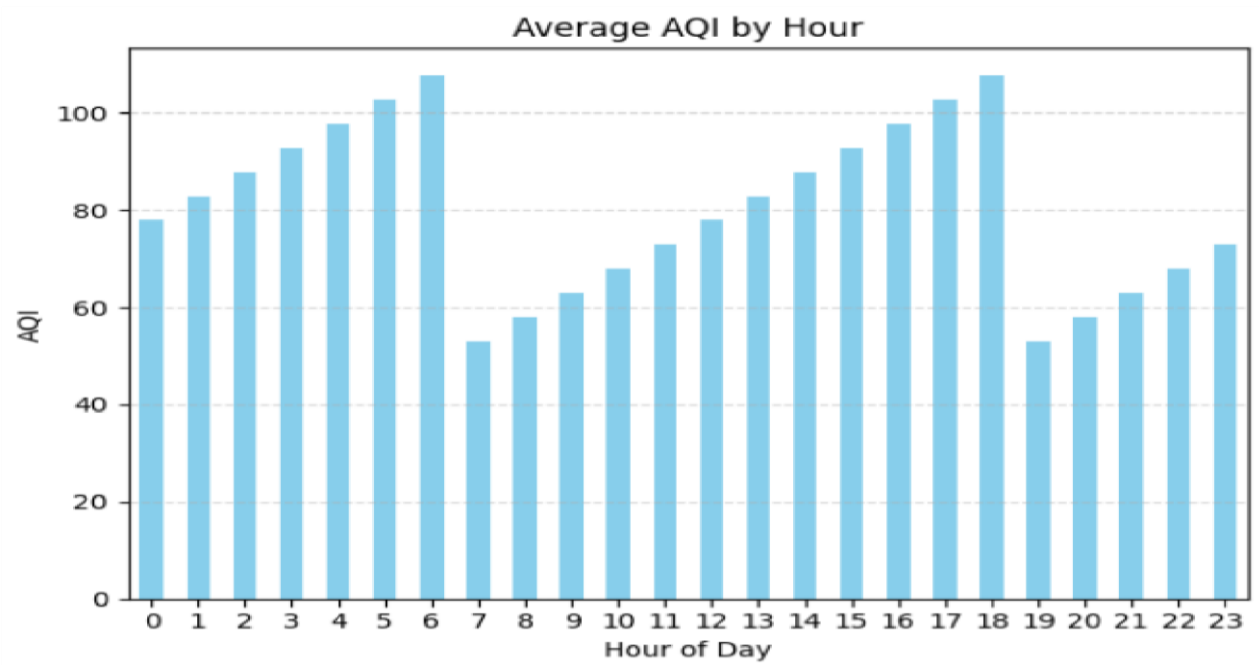
Correlation Heatmap



Average AQI by Day of Week



Average AQI by Hours of Day



5. Model Training & Evaluation

Various machine learning and deep learning models are trained to predict AQI. Models like Linear Regression, Ridge Regression, Random Forest, and LSTM are

evaluated using metrics like MAE, RMSE, and R^2. Time Series Split is used to maintain temporal integrity.

Result After each Model Training & Evaluation

1. LightGBM

The accuracy of the LightGBM model after training is 92.73%.

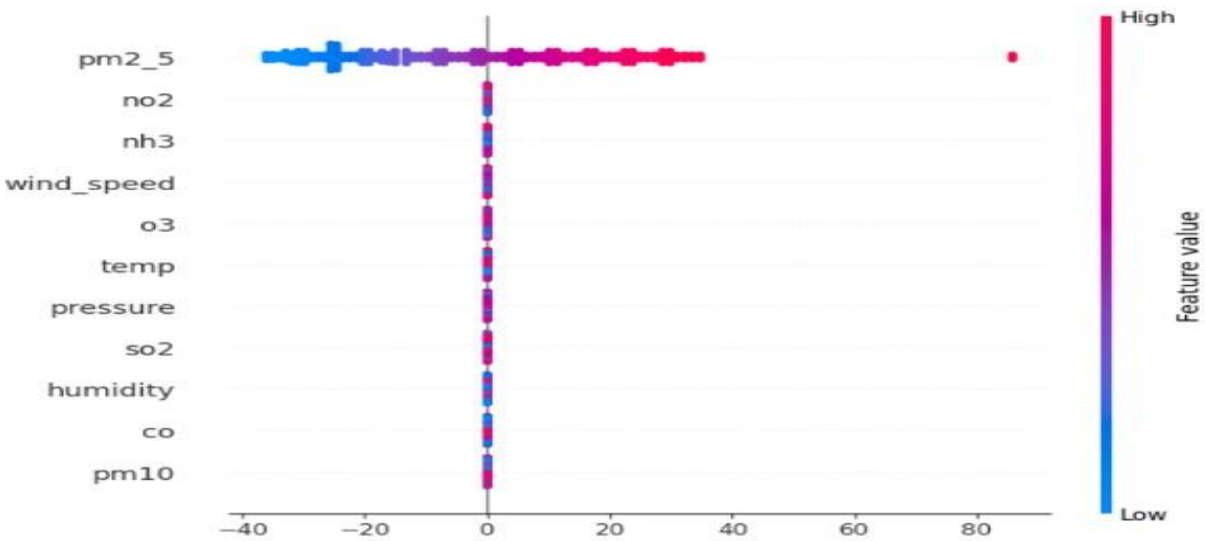
Evaluation Metrics The

value of MAE is 1.92.

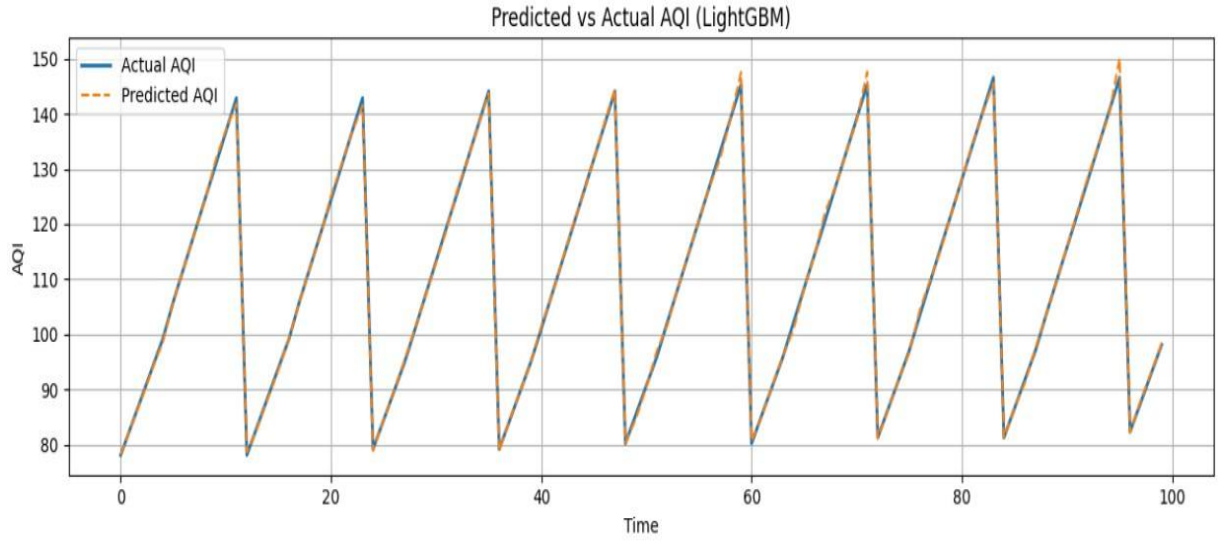
The value of RMSE is 5.98.

The value of R2 score is 0.93.

Using SHAP explainability, it was observed that the feature contributing most to the AQI prediction is PM2.5.



This figure indicates that the model is well-generalized, as all data points closely follow the reference line.



2. XGBoost

After training, the XGBoost model achieved an accuracy of 98.92%.

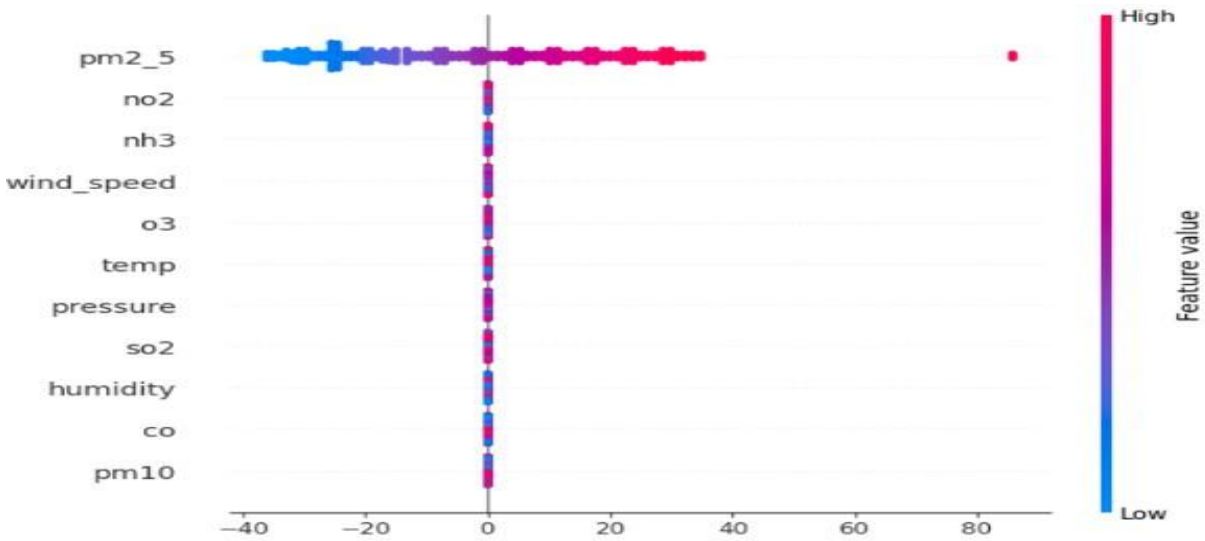
Evaluation Metrics The

value of MAE is 0.40.

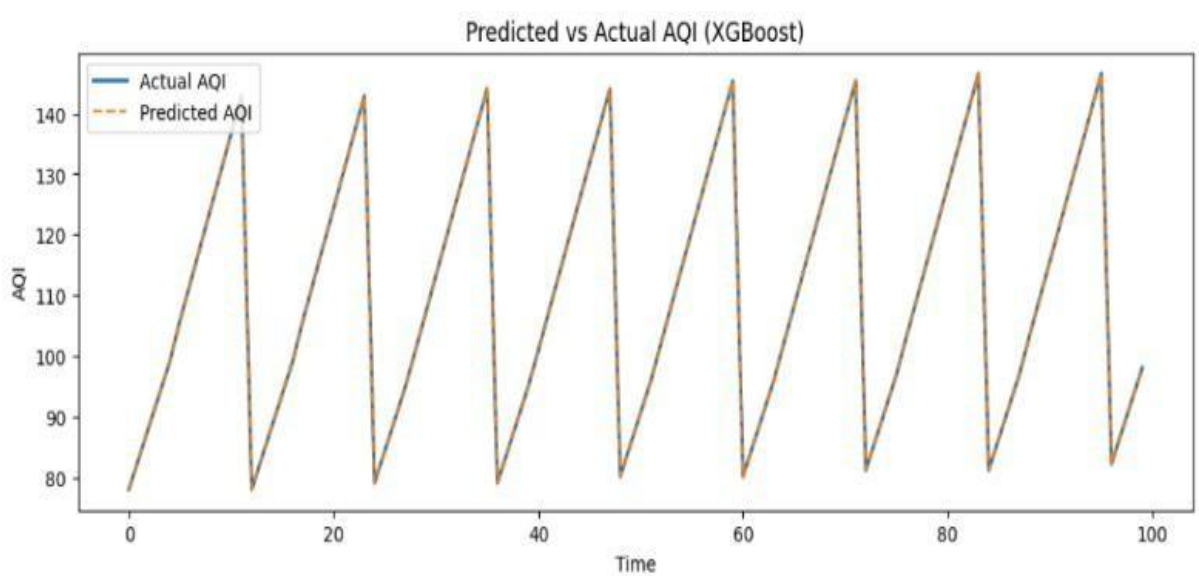
The value of RMSE is 1.15.

The value of R2 score is 0.99.

Using SHAP explainability, it was observed that the feature contributing most to the AQI prediction is PM2.5.



This figure indicates that the model is well-generalized, as all data points closely follow the reference line.



3. RIDGE REGRESSION

After training, the Ridge Regression model achieved an accuracy of 93.77%.

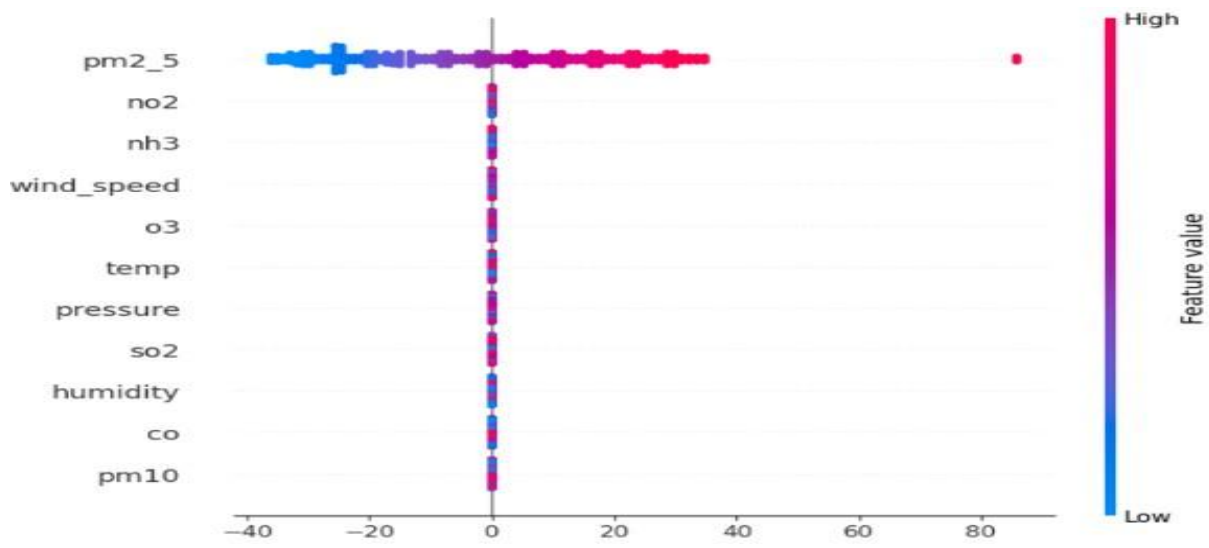
Evaluation Metrics The

value of MAE is 0.40.

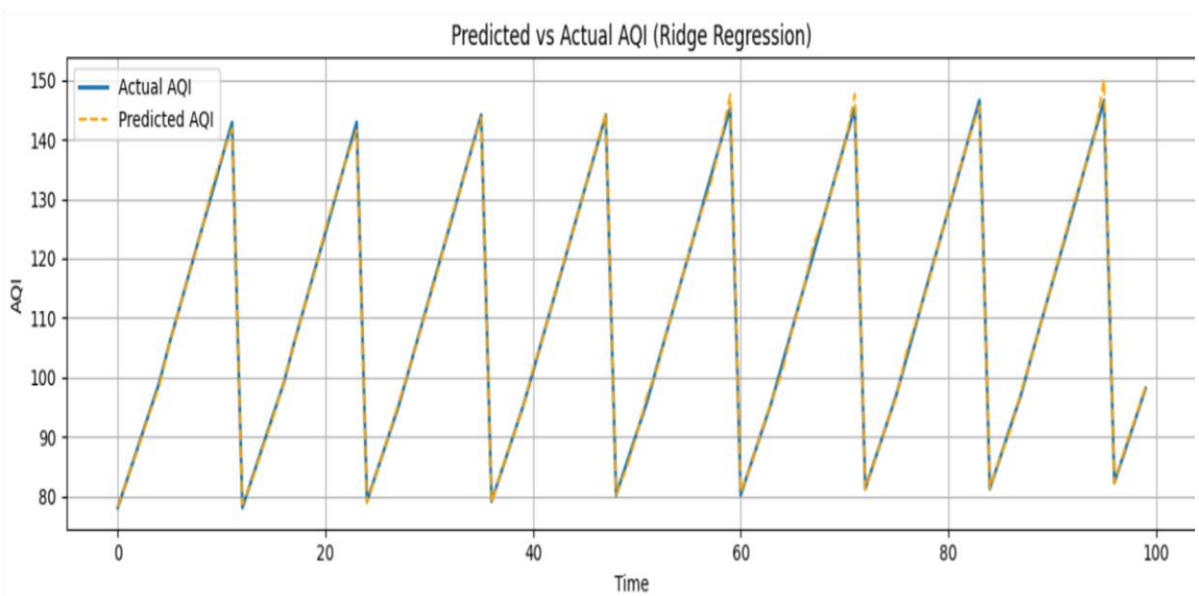
The value of RMSE is 1.15.

The value of R2 score is 0.99.

Using SHAP explainability, it was observed that the feature contributing most to the AQI prediction is PM2.5.



This figure indicates that the model is well-generalized, as all data points closely follow the reference line.



4. LINEAR REGRESSION

After training, the Linear Regression model achieved an accuracy of 95.85%.

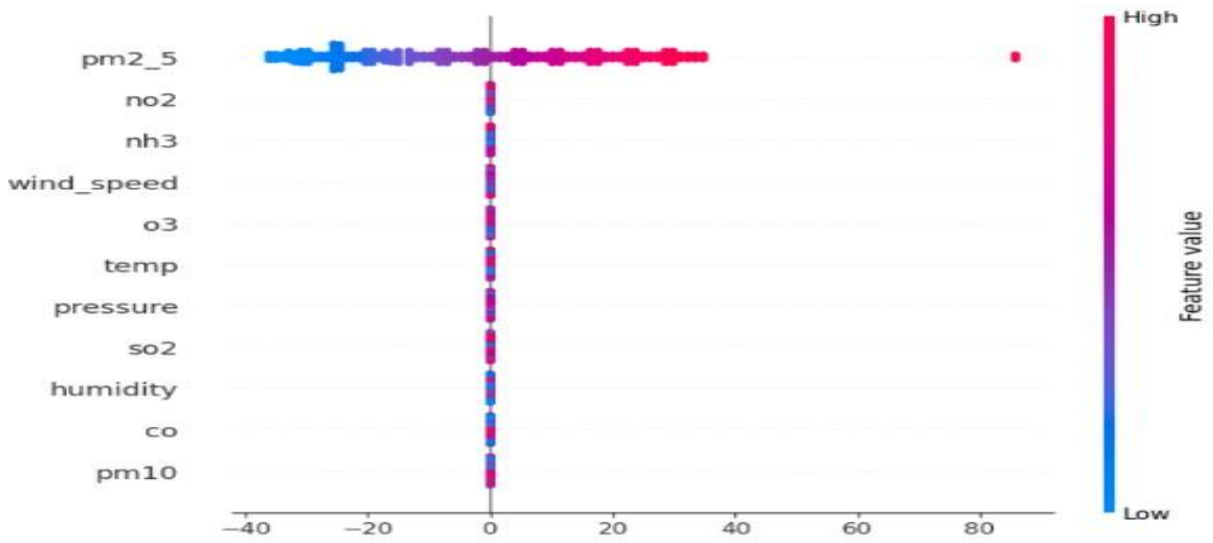
Evaluation Metrics

The value of MAE is 1.05.

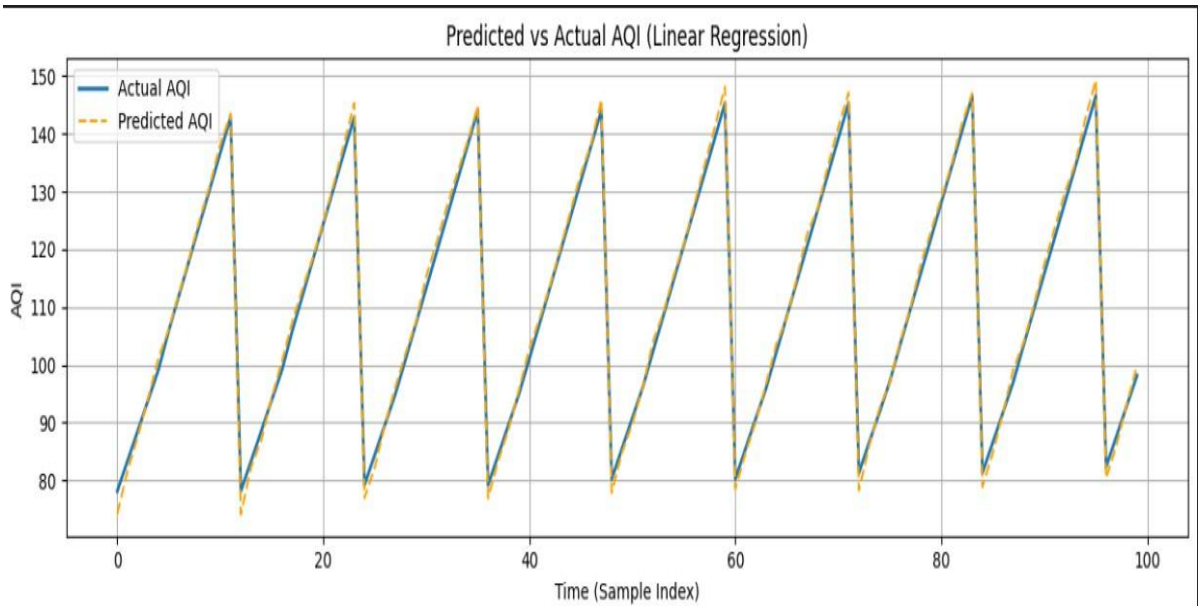
The value of RMSE is 3.62.

The value of R2 score is 0.96.

Using SHAP explainability, it was observed that the feature contributing most to the AQI prediction is PM2.5.



This figure indicates that the model is well-generalized, as all data points closely follow the reference line.



5. LSTM

After training, the LSTM model achieved an accuracy of 99.74%.

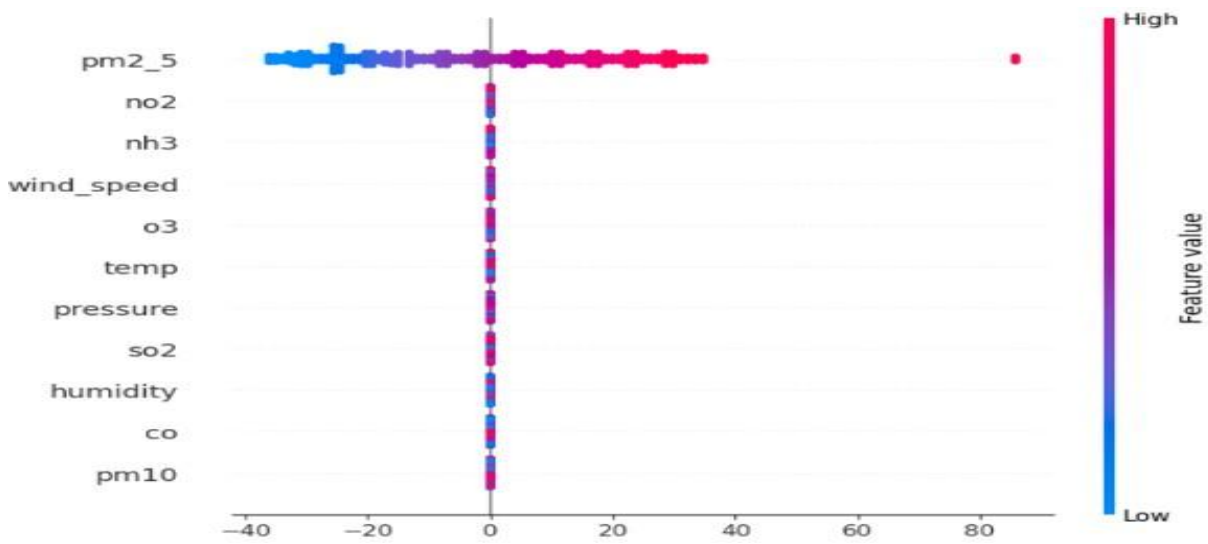
Evaluation Metrics The

value of MAE is 0.64.

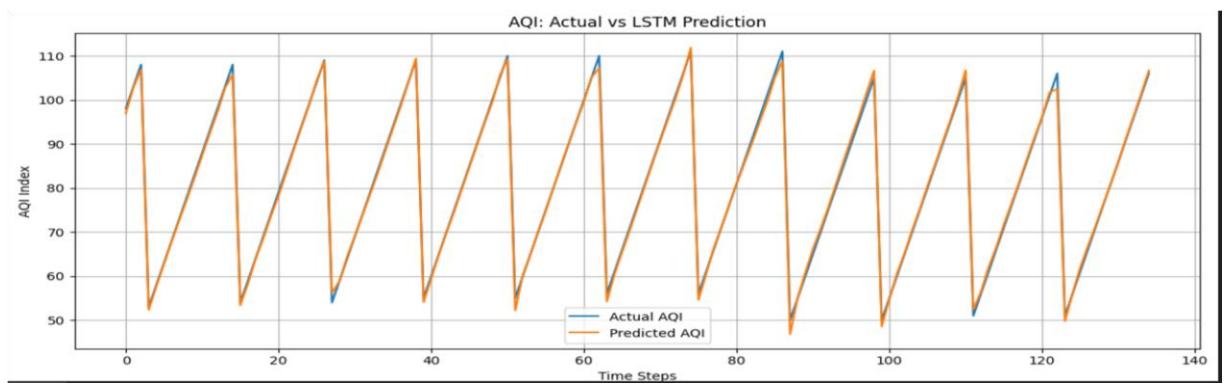
The value of RMSE is 0.89.

The value of R2 score is 1.00.

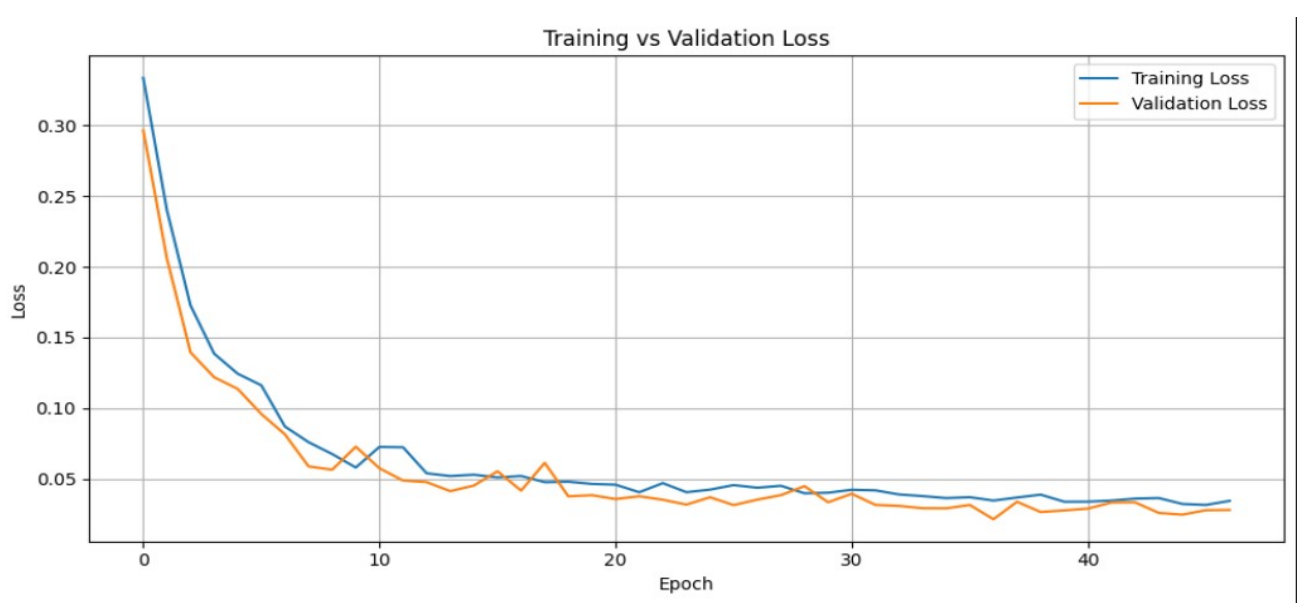
Using SHAP explainability, it was observed that the feature contributing most to the AQI prediction is PM2.5.



This figure indicates that the model is well-generalized, as all data points closely follow the reference line.



Training versus Validation loss is given by



6. MODEL TESTING

1. LightGBM

The accuracy of the LightGBM model after testing is 99.5%.

2. XGBoost

The accuracy of the XGBoost model after testing is 98.7%.

3. LINEAR REGRESSION

The accuracy of the Linear Regression model after testing is 98.7%.

4. RIDGE REGRESSION

The accuracy of the Ridge Regression model after testing is 98.7%.

5. LSTM

The accuracy of the LSTM model after testing is 98.7%.

7. Feature Store & Model Registry

A feature store (e.g., Hopsworks or Vertex AI) is used to store preprocessed features for reuse during training and inference. Trained models are saved in a model registry for version control and easy deployment.

8. CI/CD Automation

Continuous Integration and Continuous Deployment (CI/CD) pipelines are set up using GitHub Actions. These automate:

- Hourly: Feature generation from APIs
- Daily: Model retraining with the latest

data The code is given by

```
name: Hourly AQI Logger

on:
  schedule:
    - cron: '0 * * * *'  # Every hour
  workflow_dispatch:

jobs:
  log-aqi:
    runs-on: ubuntu-latest

    steps:
      - name: ✅ Checkout Repo
        uses: actions/checkout@v4

      - name: 🐍 Set Up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: 📦 Install Python Dependencies
        run: pip install pandas requests

      - name:    Collect and Merge AQI Data
        run: |
          python - <<EOF
          import requests
          import pandas as pd
          import datetime
          import os
          from glob import glob

          IR_KEY = "a1c4ea3c-c116-43fb-9d98-81004cf3f771"
          WEATHER_KEY = "ee88310481e99a7843bdedc0cca27fd1"

          LAT = 24.8607
          LON = 67.0011
          CITY = "Karachi"
          COUNTRY = "PK"

          def get_iqair_aqi():
```

```
    try:
        url = f"http://api.airvisual.com/v2/nearest_city?lat={LAT}&lon={LON}&key={IR_KEY}"
        r = requests.get(url)
        r.raise_for_status()
        data = r.json()
        pollution = data["data"]["current"]["pollution"]
        return {
            "aqi_index": pollution.get("aqius")
        }
    except Exception as e:
        print("IQAir AQI Error:", e)
        return None

def get_openweather_air_pollution():
    try:
        url =
f"http://api.openweathermap.org/data/2.5/air_pollution?lat={LAT}&lon={LON}&appid={WEATHER_KEY}"
        r = requests.get(url)
        r.raise_for_status()
        data = r.json()
        components = data["list"][0]["components"]
        return {
            "pm2_5": components.get("pm2_5"),
            "pm10": components.get("pm10"),
            "co": components.get("co"),
            "no2": components.get("no2"),
            "o3": components.get("o3"),
            "so2": components.get("so2"),
            "nh3": components.get("nh3")
        }
    except Exception as e:
        print("OpenWeather Pollutants Error:", e)
        return None

def get_weather():
    try:
        url =
f"http://api.openweathermap.org/data/2.5/weather?q={CITY},{COUNTRY}&appid={WEATHER_KEY}&units=metri
c"
        r = requests.get(url)
        r.raise_for_status()
        data = r.json()
        return {
            "temp": data["main"]["temp"],
            "humidity": data["main"]["humidity"],
            "pressure": data["main"]["pressure"],
            "wind_speed": data["wind"]["speed"]
        }
```

```
        except Exception as e:
            print("Weather Error:", e)
            return None

    def get_forecast_features():
        try:
            url =
f"http://api.openweathermap.org/data/2.5/forecast?lat={LAT}&lon={LON}&appid={OPENWEATHER_KEY}&units
=metric"
            r = requests.get(url)
            r.raise_for_status()
            data = r.json()
            forecasts = data["list"]
            out = {}
            for i, entry in enumerate(forecasts[:24]):
                out[f"forecast_{i+1}_temp"] = entry["main"]["temp"]
                out[f"forecast_{i+1}_humidity"] = entry["main"]["humidity"]
                out[f"forecast_{i+1}_pressure"] = entry["main"]["pressure"]
                out[f"forecast_{i+1}_wind_speed"] = entry["wind"]["speed"]
            return out
        except Exception as e:
            print("Forecast Error:", e)
            return {}

    def collect_hourly_data():
        now = datetime.datetime.utcnow()
        date_str = now.strftime("%Y-%m-%d")
        time_str = now.strftime("%H:%M:%S")
        hour_of_day = now.hour
        day_of_week = now.weekday()
        is_weekend = 1 if day_of_week >= 5 else 0

        folder = f"data/{date_str}"
        os.makedirs(folder, exist_ok=True)
        csv_path = f"{folder}/karachi.csv"

        aqi_data = get_iqair_aqi()
        pollutants = get_openweather_air_pollution()
        weather = get_weather()
        forecast = get_forecast_features()

        if not aqi_data or not pollutants or not weather or not forecast:
            print("❌ Incomplete data. Skipping logging.")
            return

        row = {
            "date": date_str,
            "time": time_str,
```

```
        "hour_of_day": hour_of_day,
        "day_of_week": day_of_week,
        "is_weekend": is_weekend,
        **aqi_data,
        **pollutants,
        **weather,
        **forecast
    }

    df = pd.DataFrame([row])
    if os.path.exists(csv_path):
        df.to_csv(csv_path, mode="a", header=False, index=False)
    else:
        df.to_csv(csv_path, index=False)

    print(f"✅ Data saved to {csv_path}")

def merge_all_data():
    files = glob("data/*/karachi.csv")
    if not files:
        print("No files to merge.")
        return
    all_data = pd.concat([pd.read_csv(f) for f in files], ignore_index=True)
    os.makedirs("data1", exist_ok=True)
    all_data.to_csv("data1/merged_karachi_aqi.csv", index=False)
    print("✅ Merged file saved to data1/karachi_aqi.csv")

collect_hourly_data()
merge_all_data()
EOF

- name: 📂 Commit and Push
  env:
    GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
  run: |
    git config --global user.name "github-actions"
    git config --global user.email "actions@github.com"
    git add data/ data1/
    git commit -m "📊 Hourly AQI collected and merged - $(date -u)" || echo "No changes to commit"
    git push
```

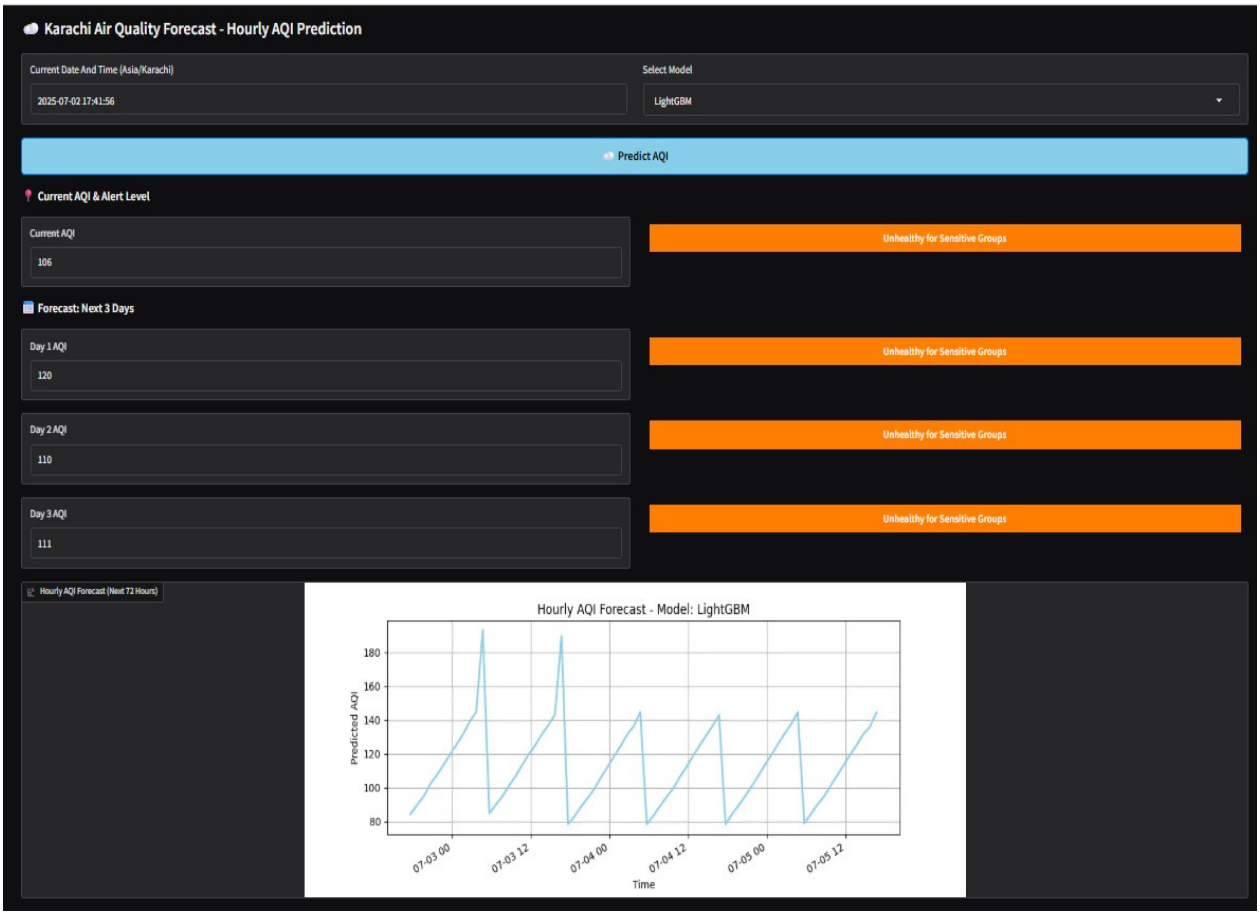
8. API Development

A RESTful API is built using FastAPI to serve real-time AQI predictions. This allows the frontend dashboard and other services to fetch predictions programmatically.

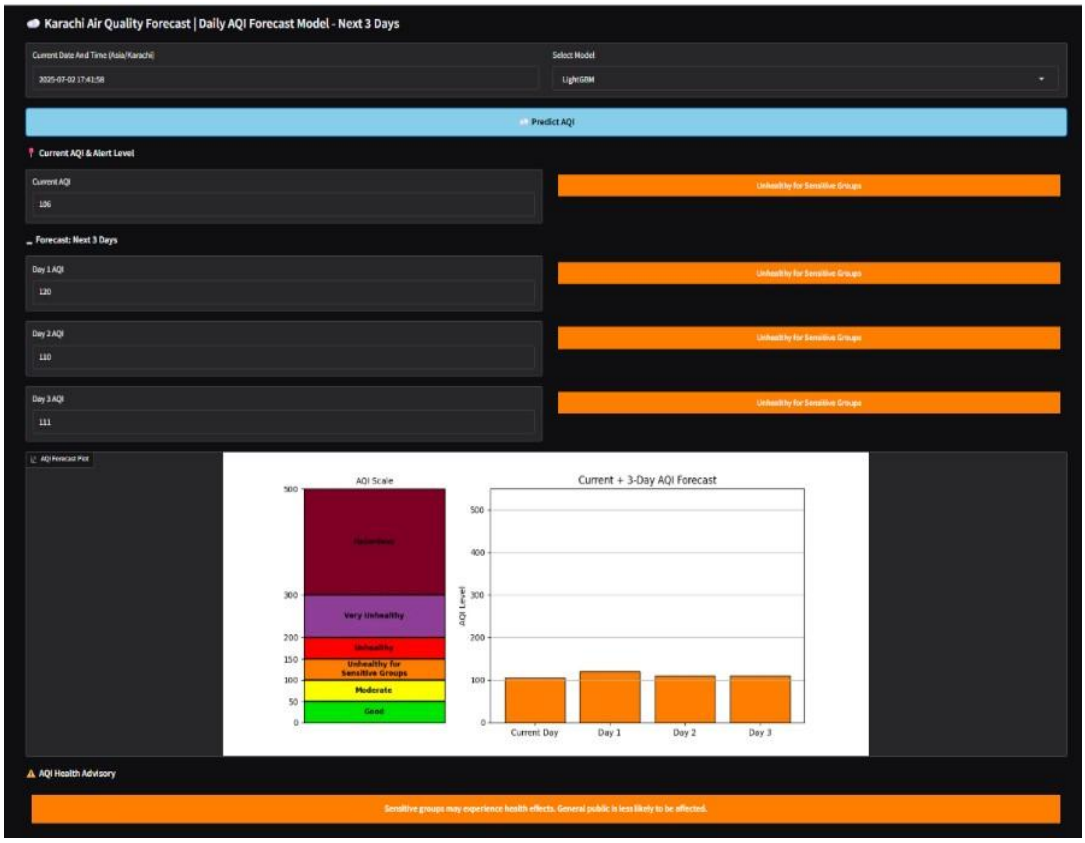
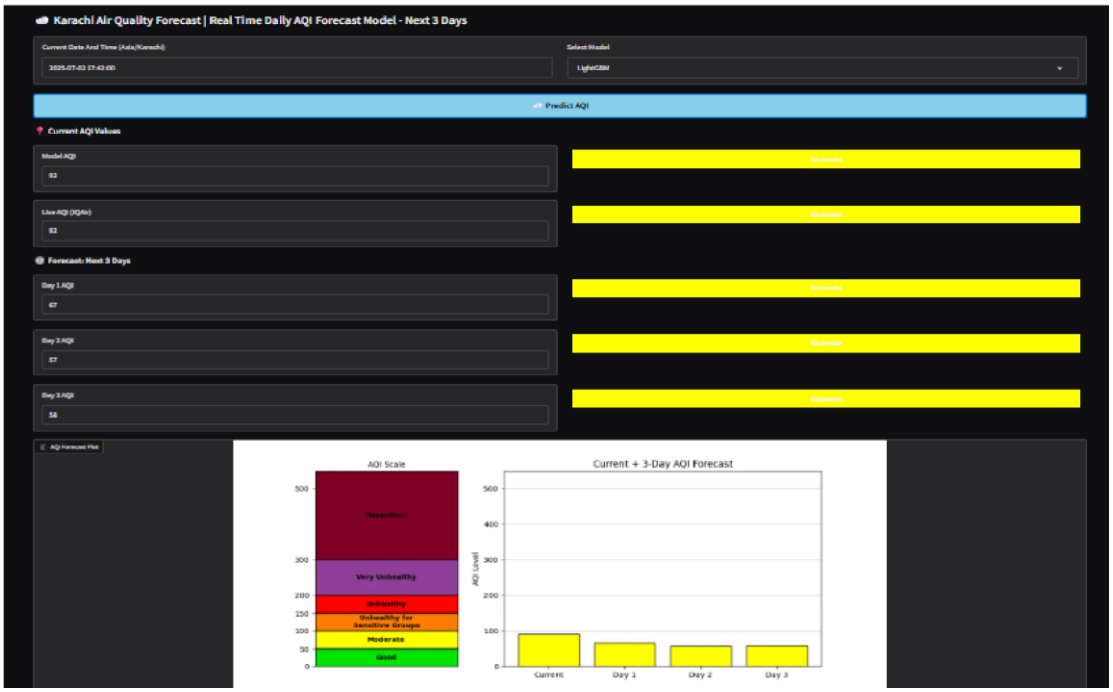
9. Dashboard

An interactive dashboard is developed using Streamlit to display real-time AQI, 3day forecasts, alert levels, and health advisories. It offers users a visual and intuitive interface to monitor air quality.

1. Using Hourly Data Based on Trained Data



2. Using Daily Data Based on Trained Data



3. Real Time Daily AQI Forecast Model – Next 3 Days (Displays current AQI from IQAir, current predicted AQI, and the forecasted AQI for the next 3 days.)